📄 Especificación de Requerimientos de Software (SRS)

Proyecto: SyncBar Audio
Estándar de Referencia: ISO/IEC/IEEE 29148:2018
Versión: 1.0
Fecha: Mayo 2026

---

1. Introducción

1.1 Propósito

El propósito de este documento es definir los requerimientos funcionales, no funcionales y de interfaz para el desarrollo de una utilidad de sistema multiplataforma (macOS y Windows) destinada a la gestión avanzada de dispositivos de audio y la creación de grupos de salida virtuales de baja latencia.

1.2 Alcance del Sistema

SyncBar Audio operará como una aplicación exclusivamente residente en la barra de menú del sistema (Menu Bar en macOS / System Tray en Windows). El sistema interactuará a bajo nivel con los subsistemas de audio nativos del hardware mediante un núcleo unificado, eliminando la necesidad de software de ruteo complejo para el usuario final.

1.3 Definiciones, Acrónimos y Abreviaturas

- **HAL:** Hardware Abstraction Layer (Capa de Abstracción de Hardware).
- **FFI:** Foreign Function Interface (Interfaz de Funciones Foráneas).
- **UID:** Unique Identifier (Identificador Único Persistente de Hardware).
- **WASAPI:** Windows Audio Session API.
- **CoreAudio:** API nativa de audio para sistemas operativos Apple macOS.

---

2. Descripción General

2.1 Perspectiva del Producto

El software adopta una arquitectura de **Núcleo Compartido** desarrollado en C++ utilizando abstracciones multiplataforma, mientras que las interfaces de usuario (UI) se mantendrán

100% nativas e independientes para cada sistema operativo para garantizar un consumo mínimo de recursos.

2.2 Funciones del Producto

- Detección dinámica y listado de dispositivos de reproducción de audio activos.
- Control individual de volumen maestro y estado de silencio (Mute) por hardware.
- Agrupamiento virtual lógico de múltiples canales de salida de audio simultáneos.
- Persistencia local de metadatos, nombres personalizados y configuraciones de usuario.

2.3 Restricciones del Diseño y la Implementación

- **Consumo de Memoria:** La interfaz de usuario no deberá superar los 50 MB de consumo de memoria RAM en estado de inactividad.
- **Sin Modificaciones del Kernel:** La app no debe requerir la instalación de KEXTs (Kernel Extensions) firmados ni alterar la integridad del sistema operativo.
- **Entorno de Ejecución:** Debe ser compatible con macOS 13+ (Apple Silicon y x86) y Windows 11 (arquitecturas de 64 bits).

2.4 Arquitectura del Proyecto

La arquitectura se divide en tres capas horizontales. El objetivo es que la lógica de "negocio" (cómo se agrupan los dispositivos, los nombres personalizados y las reglas de volumen) sea independiente del sistema operativo.

- **1. Capa de Interfaz (Nativa):** Cada sistema operativo tiene su propia interfaz. Esto garantiza que la app ocupe poca RAM y se integre perfectamente con el diseño del sistema.
    - macOS: SwiftUI (usando MenuBarExtra).
    - Windows: WinUI 3 con C#/.NET (usando TrayIcon).

- **2. Capa de Enlace (Bridge / FFI):** Permite que la interfaz (Swift o C#) llame a las funciones del núcleo (C++). Se expone una interfaz tipo C-API para que ambos sistemas puedan "ver" las funciones del motor.

- **3. Núcleo Compartido (C++ Engine):** Es el cerebro. Se encarga de:
    - Audio Logic: Cálculos de decibelios y normalización.
    - State Management: Qué dispositivos están activos o muteados.
    - HAL (Hardware Abstraction Layer): Aquí es donde el código se bifurca mediante compilación condicional para hablar con CoreAudio o WASAPI.

3. Requerimientos Específicos (ISO 29148 Structure)

3.1 Requerimientos Funcionales (RF)

Capa del Núcleo (Core Engine en C++)

- **RF-01 [Detección de Hardware]:** El sistema debe consultar el subsistema de audio nativo (CoreAudio/WASAPI) de forma reactiva ante eventos de conexión y desconexión de hardware y recuperar su UID persistente.
- **RF-02 [Sincronización de Grupos]:** El núcleo debe calcular las relaciones matemáticas de ganancia en decibelios (dB) cuando se aplique una modificación de volumen a un grupo virtual, distribuyendo la proporción de volumen equitativamente entre los dispositivos miembros.
- **RF-03 [Persistencia Local de Datos]:** El sistema debe escribir los cambios de configuración del usuario en una base de datos local SQLite utilizando un patrón de retraso (*Debounce*) para evitar ciclos de escritura excesivos durante el arrastre de sliders.

Capa de Interfaz de Usuario (UI - macOS / Windows)

- **RF-04 [Interfaz de Tipo Utilidad]:** La aplicación no debe desplegar ventanas principales en el Dock o Barra de Tareas. Debe vivir exclusivamente en la barra de estado superior (macOS) o bandeja del sistema inferior (Windows).
- **RF-05 [Edición de Etiquetas]:** La interfaz debe proveer un mecanismo visual de edición en línea (*inline editing*) para permitir al usuario renombrar la etiqueta visual de cualquier dispositivo registrado.

3.2 Requerimientos No Funcionales (RNF)

RNF-01: Eficiencia y Rendimiento de Audio

- **Descripción:** La latencia introducida por el núcleo de la aplicación al unificar las buffers de audio en los grupos virtuales no debe exceder los **10 milisegundos**.
- **Medición:** Pruebas automatizadas de bucle de retorno de audio (*loopback processing*).

RNF-02: Portabilidad y Compilación

- **Descripción:** El código fuente alojado en el directorio /core debe ser completamente agnóstico del sistema operativo, aislándose los módulos nativos mediante directivas de preprocesador (#ifdef _WIN32 y #ifdef __APPLE__).

RNF-03: Robustez de Datos

- **Descripción:** La base de datos local SQLite debe implementar integridad referencial (PRAGMA foreign_keys = ON) y operaciones en cascada (ON DELETE CASCADE) para prevenir la existencia de registros huérfanos de dispositivos eliminados.

---

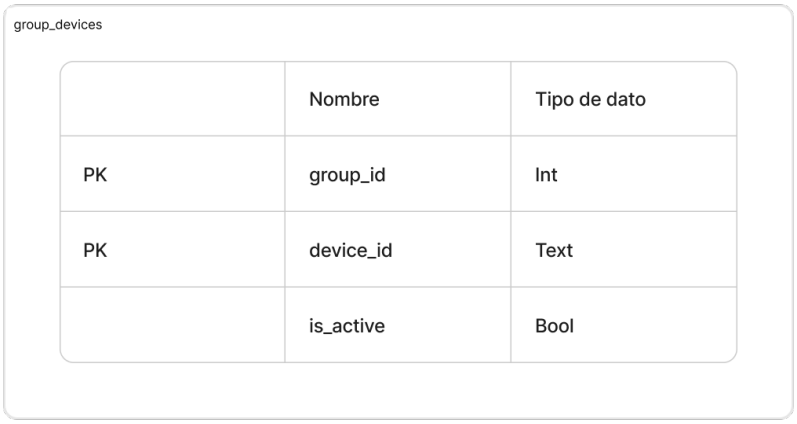
4. Interfaces del Sistema

4.1 Interfaz de Base de Datos Local (SQLite)

La persistencia de configuraciones se regirá bajo el siguiente esquema relacional estricto:

Tabla	Campo Clave	Tipo	Propósito
devices	device_uid	TEXT (PK)	Almacena configuraciones del hardware detectado.
virtual_groups	id	INTEGER (PK)	Define las entidades de agrupamiento creadas por el usuario.
group_devices	(group_id, device_uid)	Composite (PK)	Tabla intermedia que mapea la relación muchos a muchos.

4.2 Tablas de bases de datos



virtual_groups

	Nombre	Tipo de dato
PK	group_id	Int
	name	Text
	is_active	Bool

dispositivos

	Nombre	Tipo de dato
PK	device_id	Text
	custom_name	Text
	saved_volume	REAL
	is_muted	bool

5. ESTURCTURA DEL PROYECTO

```
audiosync-pro/
 ├── CMakeLists.txt                  # Configuración principal de CMake para compilar el núcleo
 ├── README.md                        # Documentación técnica del proyecto
 ├── core/                            # NÚCLEO COMPARTIDO (C++)
 │    ├── CMakeLists.txt          # Configuración de compilación para la librería estática/dinámica
 │    ├── include/                    # Archivos de cabecera (.h / .hpp)
 │    │    ├── audio_engine.h        # Clase principal del motor de audio
```

```
(JUCE/CoreAudio/WASAPI)
│   │   ├── database_mgr.h          # Gestor de persistencia SQLite
│   │   └── bridge_api.h            # La C-API expuesta para el Bridge (FFI)
│   └── src/                        # Implementación del código (.cpp)
│       ├── audio_engine.cpp        # Lógica de audio y bifurcación HAL (#ifdefs)
│       ├── database_mgr.cpp        # Consultas e inserciones de SQLite
│       └── bridge_api.cpp          # Implementación de las funciones del Bridge
├── macos/                          # CAPA DE INTERFAZ MAC (SwiftUI)
│   ├── AudioSyncPro.xcodeproj      # Proyecto de Xcode
│   └── AudioSyncPro/
│       ├── App.swift               # Punto de entrada (MenuBarExtra)
│       ├── Views/                  # Componentes de la UI (Sliders, Toggles, Pop-over)
│       └── CoreBridge/             # Archivos de enlace (.bridging-header) para mapear el Bridge C++
└── windows/                        # CAPA DE INTERFAZ WINDOWS (WinUI 3 / C#)
    ├── AudioSyncPro.sln            # Solución de Visual Studio
    └── AudioSyncPro/
        ├── App.xaml                # Configuración de la app y ciclo de vida
        ├── App.xaml.cs             # Punto de entrada (TrayIcon e inicialización de la UI)
        ├── Views/                  # Vistas de la interfaz en XAML (Estética Mica/Acrylic)
        └── CoreBridge/             # Clases C# con P/Invoke para importar las funciones de la DLL
```